



Small
Artificial
RAM-restricted
Deep
Intelligent
Neural
Engine

SARDINE: Chess AI Under Hard Memory Limits

Small Artificial RAM-restricted Deep Intelligent Neural Engine

ICTP Internship · July 2026

The Hardware Ceiling

Resource	Wio Terminal	Why it matters
CPU	120 MHz (SAMD51)	Eval latency caps search depth
RAM	192 KB	TT must dominate; accumulators ~16 KB
Flash	512 KB	Weights ~10% after int8 + L1 prune
Move budget	~1 second	Urusov ESP32 baseline ~20 kNps (no NNUE)
Target	≥ 1700 Elo	Dog (ESP32) proves feasibility at ~320 KB RAM

192 KB RAM

< one JPEG image

What the Kaggle Challenge Taught Us

FIDE & Google Efficient Chess AI — 5 MiB RAM, 64 KiB binary cap

NNUE won at the top

Micro int8 nets + king mirroring + geometric zeros beat pure HCE for peak Elo.

RAM = search depth

Winners gutted history/hash tables to free 512 KB–1 MiB for the transposition table.

Buckets help

2nd place Approvers: 8 output buckets by piece count in ~45 KB net.

C over C++

Cfish base — lower runtime overhead; informs SARDINE's planned C port.

Design Principles We Adopted



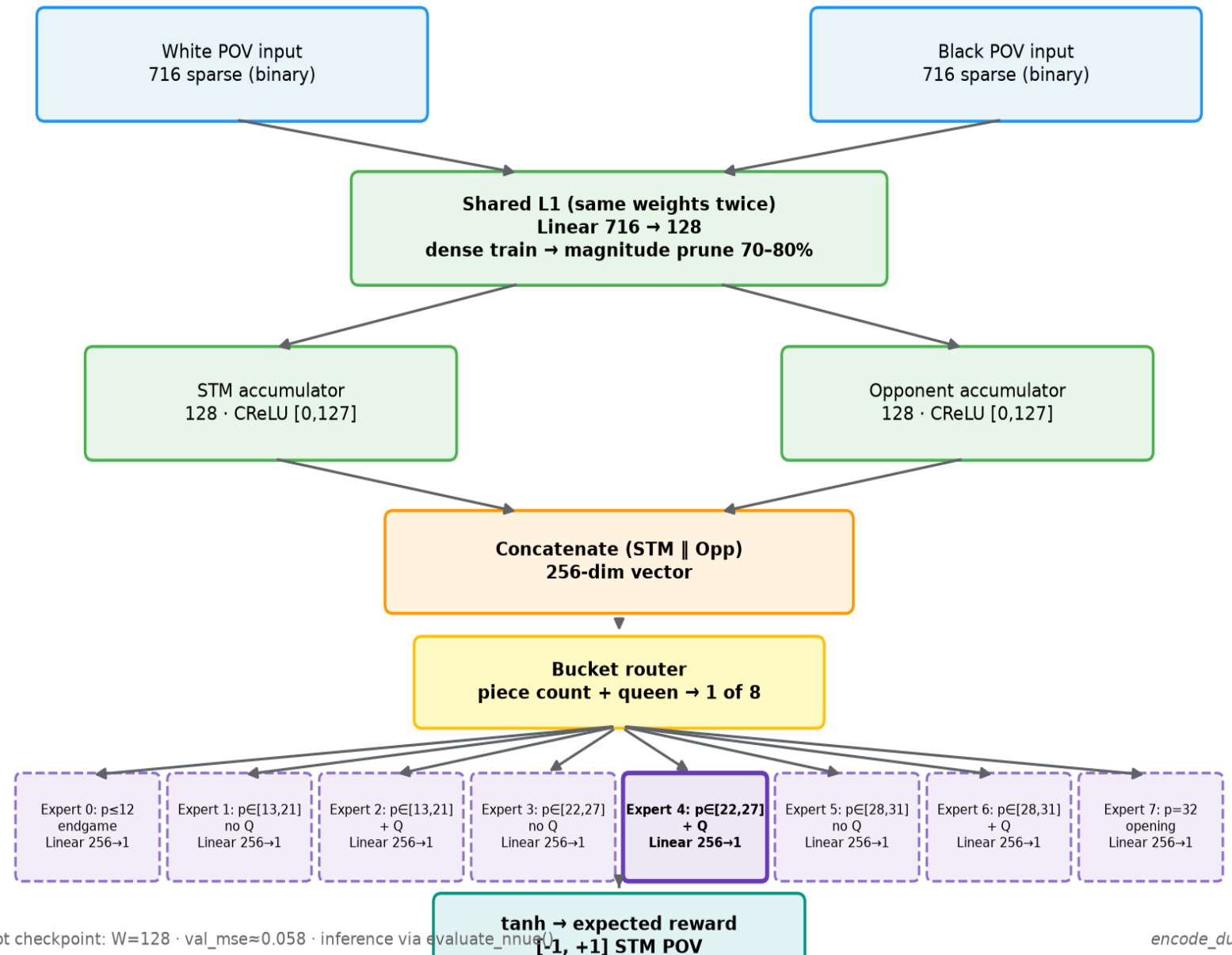
Principle	Kaggle (5 MiB)	SARDINE (192 KB)
Eval	Micro NNUE, int8, buckets	844-dim bucketed NNUE + MoE (8 experts)
RAM layout	TT 512 KB – 1 MiB	TT-dominant 128–160 KB
Compression	King mirror, pawn-rank zeros	Same + gradual L1 prune 70–80%
Runtime	Cfish / C port	C engine after PC bring-up
Search	LMR, null-move, SPSA	Alpha-beta v0.3 → full stack + SPSA

Evaluation: Bucketed NNUE + MoE

- 844 sparse features (716 base + 128 tactical) · dual perspective
- Shared L1: 844
→ W (W=128) · CReLU hidden · tanh LUT
→ expected reward [-1, +1]
- 8 expert heads routed by piece count (+ queen-split at $p \leq 12$)

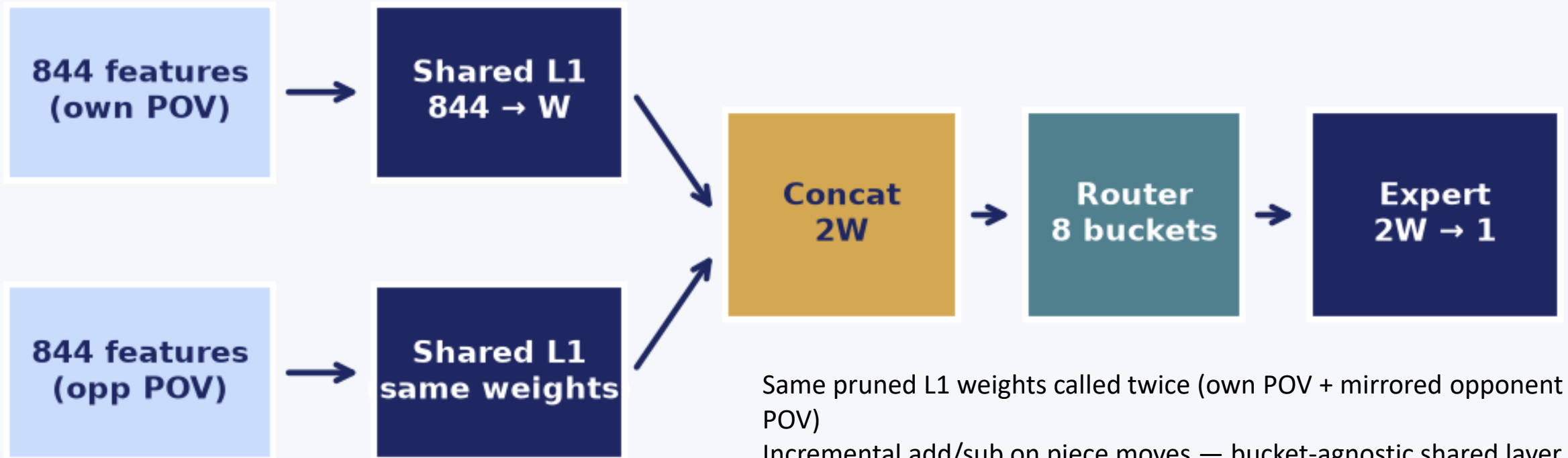
SARDINE Bucketed NNUE

716 sparse features · shared L1 W=128 · 8 expert heads · 93,832 trainable params



Shared L1 Accumulator

Incremental add/sub on moves · full refresh on king centre-file crossing



Same pruned L1 weights called twice (own POV + mirrored opponent POV)

Incremental add/sub on piece moves — bucket-agnostic shared layer
Lazy full refresh only when the king crosses the centre file

Alpha-Beta Search (v0.3)

Implemented now

Fixed-depth negamax alpha-beta
Capture quiescence at leaf nodes
MVV-LVA move ordering
Perft validated: d5 = 4,865,609 nodes
HCE default; NNUE via --eval nnue

Roadmap

Futility pruning + LMR + null-move
Lazy evaluation (paired with accumulators)
Iterative deepening with stable TT
Killer moves (depth > 4)
SPSA search-parameter tuning

How We Estimate Strength: Stockfish ACPL Gate

1

Engine self-play

N games at fixed depth (currently 16)

2

Stockfish analysis

100 ms/move — judges each played move

3

ACPL

Average centipawn loss vs Stockfish best move

4

Heuristic Elo

$\text{Elo} \approx 2855 - \text{ACPL} \times 10$ (floor 400, not FIDE)

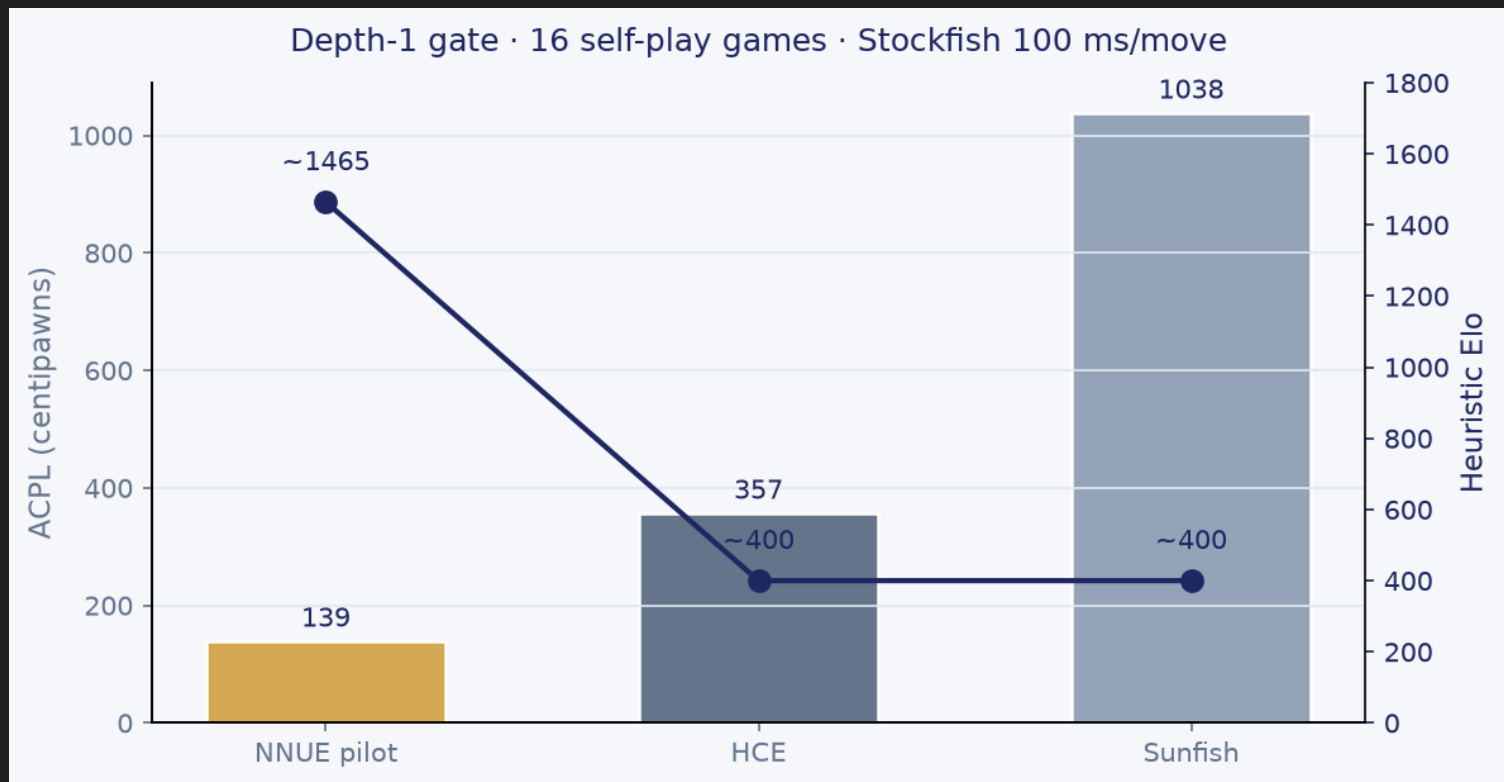
5

Final gate

cutechess-cli head-to-head for ≥ 1700 Elo (future)



Early Results (16 self-play games, depth 1)



NNUE pilot ~1465 Elo (ACPL 139) — smoke net, not production

HCE ~400 Elo (ACPL 357) — high variance, gross blunders

Sunfish ~400 Elo (ACPL 1038) — calibration reference

Pilot val_mse 0.056 · production

Lichess+Lc0 train pending

Status & Next Steps

- ✓ 844-dim feature encoder (dual POV, 8 buckets, 94 tests)
- ✓ Search v0.3 (alpha-beta, quiescence, MVV-LVA, perft)
- ✓ NNUE pilot wired to engine (--eval nnue)
- ✓ Stockfish ACPL gate + label_positions.py scaffold
- Lichess PGN → FEN + production nnue-pytorch training
- C port to Wio + incremental accumulators on device
- Full search stack + cuteshess Elo gate (≥ 1700)
- Plays online



Small
Artificial
RAM-restricted
Deep
Intelligent
Neural
Engine

Thank you — questions?